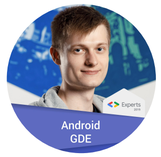
Google Developers

Experts

Building Async Mechanisms

Using the Kotlin Coroutines API, in Android



Filip Babić
@filbabic

Contents

- Why Coroutines?
- What are Coroutines?
- Suspend vs. Blocking
- Implementing Coroutines
- Effective usage
- Best Practices
- Internal Works

Why Coroutines?

Moving from Rx and Callbacks

Provides a clear, **sequential** and **understandable** syntax, to handle complex asynchronous work.

Performant (out of the box), **easy to use**, and **simple to learn***.

Solves all the problems as other mechanisms, while being straightforward.

What are Coroutines?

A blast from the past

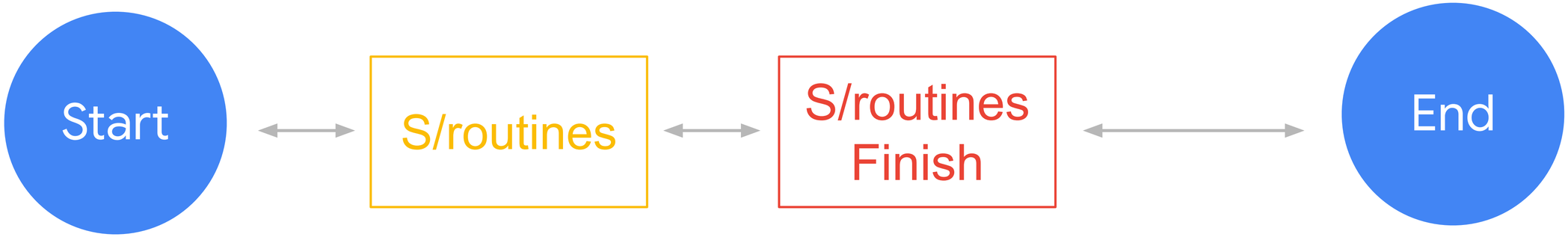
Fairly old concept (dates back to the 60s) in computer programming.

A special kind of **system routines** (think functions), which can be **suspended** and **resumed** at any point in time.

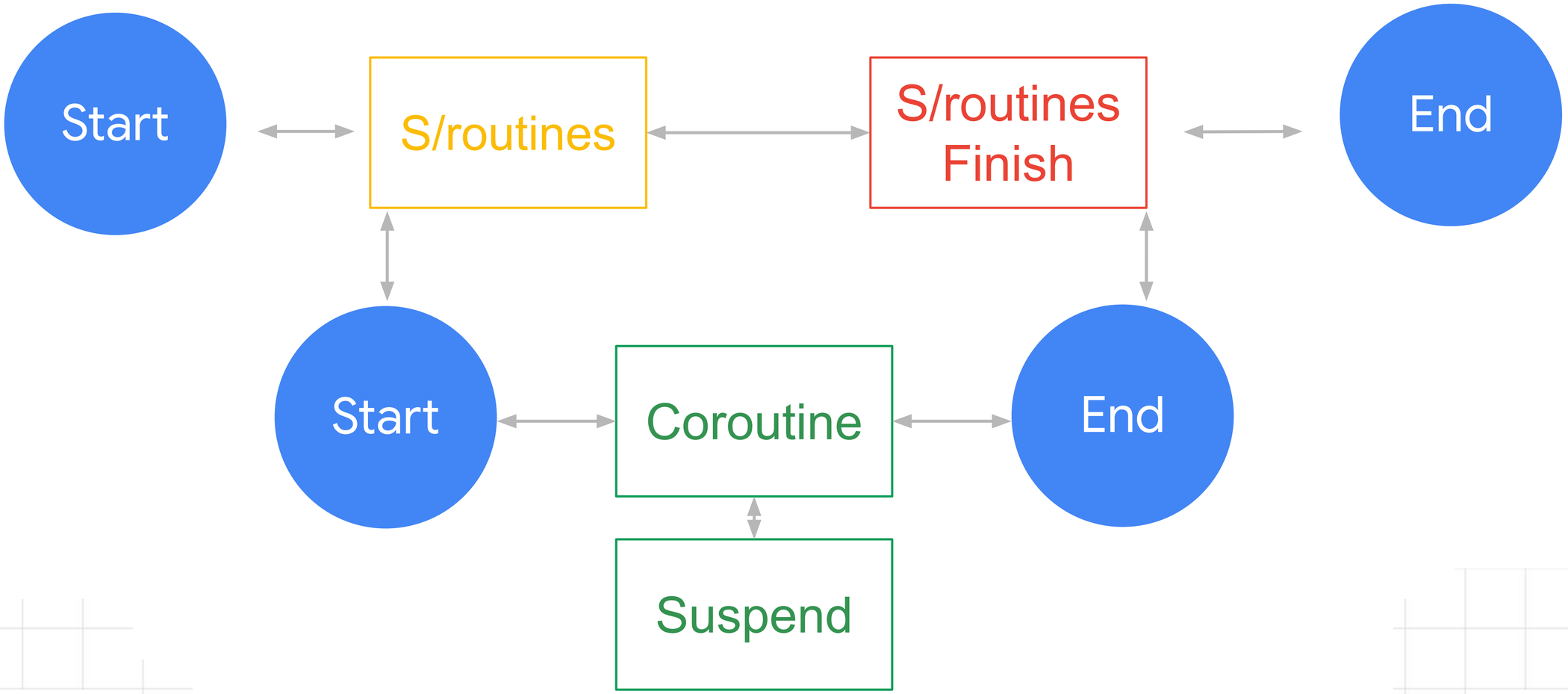
Can run in **parallel**, with other routines - hence the term (co)routine.

Routine lifecycle

Simplified



Routine lifecycle (w/ coroutines)



Generalized type of subroutines, which can be suspended and resumed at any point in time.

What's in the API

Most important functions

Launch - Basic **coroutine builder**, returns a **Job** (piece of work you can cancel)

Async/Await - Advanced **coroutine builder**, returns a **Deferred** value (also a **Job**), await to get the result

WithContext - value producing suspendable function

Other Important Constructs

Internal components

CoroutineScope - Lifecycle component used to start and confine coroutines

CoroutineContext - Set of unique elements, dictates rules for a given coroutine (Lifecycle, Exception Handling, Threading)

Suspend - Modifier which marks that the function can be suspended

```
GlobalScope.launch(context = Dispatchers.Main) {
    val userId = withContext(Dispatchers.IO) { getUserId() }

    println(userId)
}
```

```
GlobalScope.launch(context = Dispatchers.Main) {
    val userId = withContext(Dispatchers.IO) { getUserId() }

    println(userId)
}
```

```
GlobalScope.launch(context = Dispatchers.Main) {
    val userId = withContext(Dispatchers.IO) { getUserId() }

    println(userId)
}
```

```
GlobalScope.launch(context = Dispatchers.Main) {
    val userId = withContext(Dispatchers.IO) { getUserId() }

    println(userId)
}
```

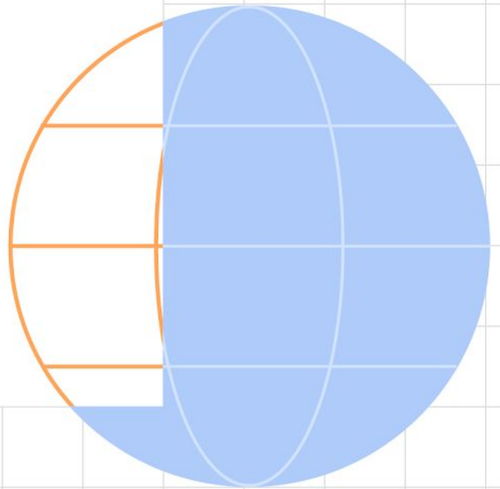
```
GlobalScope.launch(context = Dispatchers.Main) {
    val userId = withContext(Dispatchers.IO) { getUserId() }

    println(userId)
}
```

```
GlobalScope.launch(context = Dispatchers.Main) {
    val userId = withContext(Dispatchers.IO) { getUserId() }

    println(userId)
}
```

Experts

Suspend vs. Blocking

The power behind Coroutines



```
private fun doHeavyWork() {
    val expensiveResult = getExpensiveResult()

    if (isValid(expensiveResult)) {
        // do some work
    } else {
        // show an error
    }
}
```

```
private fun doHeavyWork() {
    val expensiveResult = getExpensiveResult()

    if (isValid(expensiveResult)) {
        // do some work
    } else {
        // show an error
    }
}
```

```
private suspend fun doHeavyWork() {
    val expensiveResult = getExpensiveResult()

    if (isValid(expensiveResult)) {
        // do some work
    } else {
        // show an error
    }
}
```

```
private suspend fun getExpensiveResult() =
        withContext(Dispatchers.IO) {
            // return some expensive value
        }
```

```
private suspend fun doHeavyWork() {
    val expensiveResult = getExpensiveResult()

    if (isValid(expensiveResult)) {
        // do some work
    } else {
        // show an error
    }
}
```

```
private suspend fun getExpensiveResult() =
        withContext(Dispatchers.IO) {
            // return some expensive value
        }
```

Separate Thread

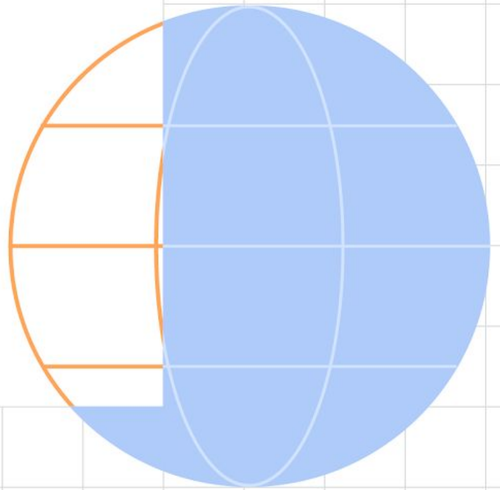
```
private suspend fun doHeavyWork() {
    val expensiveResult = getExpensiveResult()

    if (isValid(expensiveResult)) {
        // do some work
    } else {
        // show an error
    }
}
```

```
private suspend fun doHeavyWork() {
    val expensiveResult = getExpensiveResult()

    if (isValid(expensiveResult)) {
        // do some work
    } else {
        // show an error
    }
}
```

Experts

Time to get our hands on some code!

Implementing Coroutines in Android



Disclaimer!!!

“How can I do it myself?”

Disclaimer!!!

~~“How can I do it myself?”~~

How can I do it without effort?

Starting a coroutine

- **CoroutineScope** - Lifecycle
- **CoroutineContext** - rules
- **CoroutineBuilder** - type of created coroutine

```
dependencies {
    implementation "androidx.lifecycle:lifecycle-viewmodel-ktx:+"
}
```

```
class UserViewModel(private val repo: UserRepository) : ViewModel() {
  private fun registerUser(userData: UserData) {
    viewModelScope.launch {
      ...
    }
  }
}
```

```
class UserViewModel(private val repo: UserRepository) : ViewModel() {
  private fun registerUser(userData: UserData) {
    viewModelScope.launch {
      ...
    }
  }
}
```

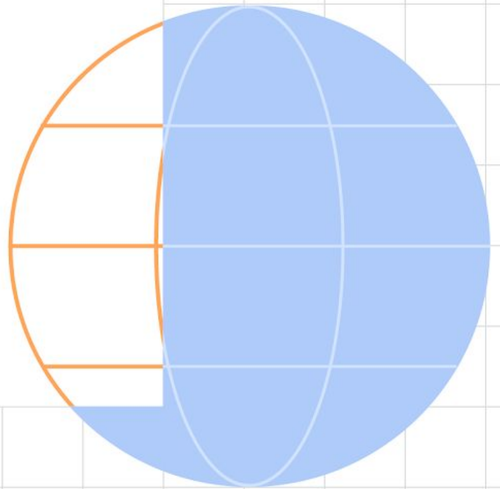
```
class UserViewModel(private val repo: UserRepository) : ViewModel() {
  private fun registerUser(userData: UserData) {
    viewModelScope.launch {
      ...
    }
  }
}
```

```
class UserViewModel(private val repo: UserRepository) : ViewModel() {
  private fun registerUser(userData: UserData) {
    viewModelScope.launch {
      val data = repo.registerUser(userData)

      // check if the data is a Success or Failure
    }
  }
}
```

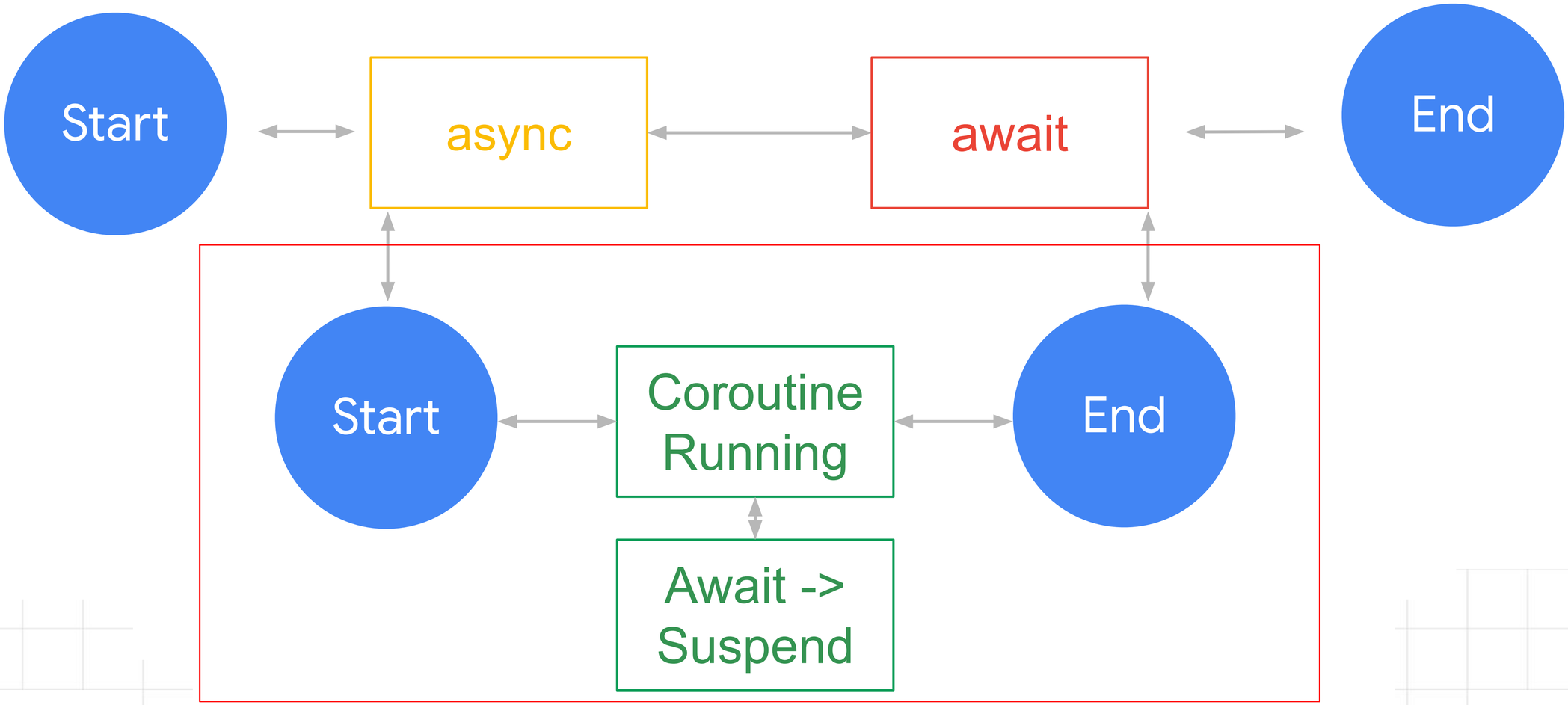
Experts

Effective usage

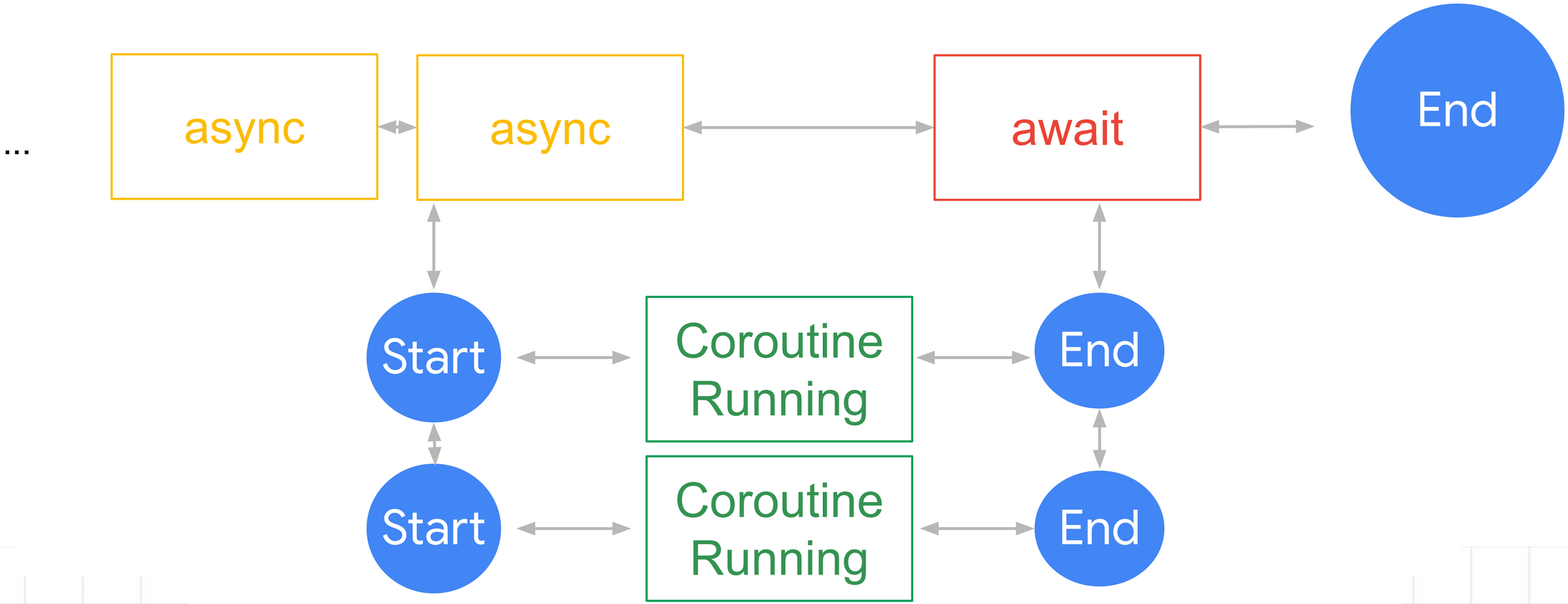
Implementing Coroutines in Android



Using Async


Start
async
await
End
Start
Coroutine
Running
End
Await ->
Suspend

(Properly) Using Async

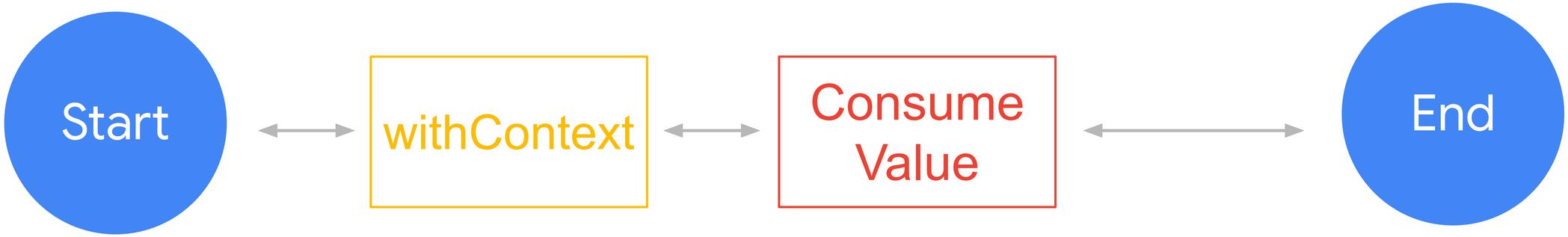


```kotlin
private suspend fun getUserProfile(userId: String): UserProfile {
  val userDeferred = async(Dispatchers.IO) { apiService.getUserById(userId) }
  val postsDeferred = async(Dispatchers.IO) { apiService.getAllPosts(userId) }
  val featuredDeferred = async(Dispatchers.IO) { apiService.getFeaturedPosts(userId) }

  return UserProfile(userDeferred.await(), postsDeferred.await(), featuredDeferred.await())
}
```

async/await is best used for multiple, parallel, value fetching. For single, or sequential-like value returns, prefer **withContext**.

```
// somewhere in the repo
override suspend fun getUserProfile(data): UserProfile =
        withContext(Dispatchers.IO) {
            apiService.getUserProfile()
        }
```

Using WithContext

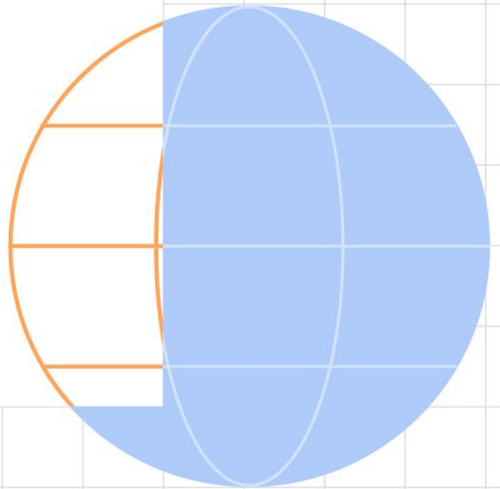


To bridge threads and sync and async work, use **withContext.** It executes a block of code in one dispatcher, returning the value to the **call site**, in the **original dispatcher**.

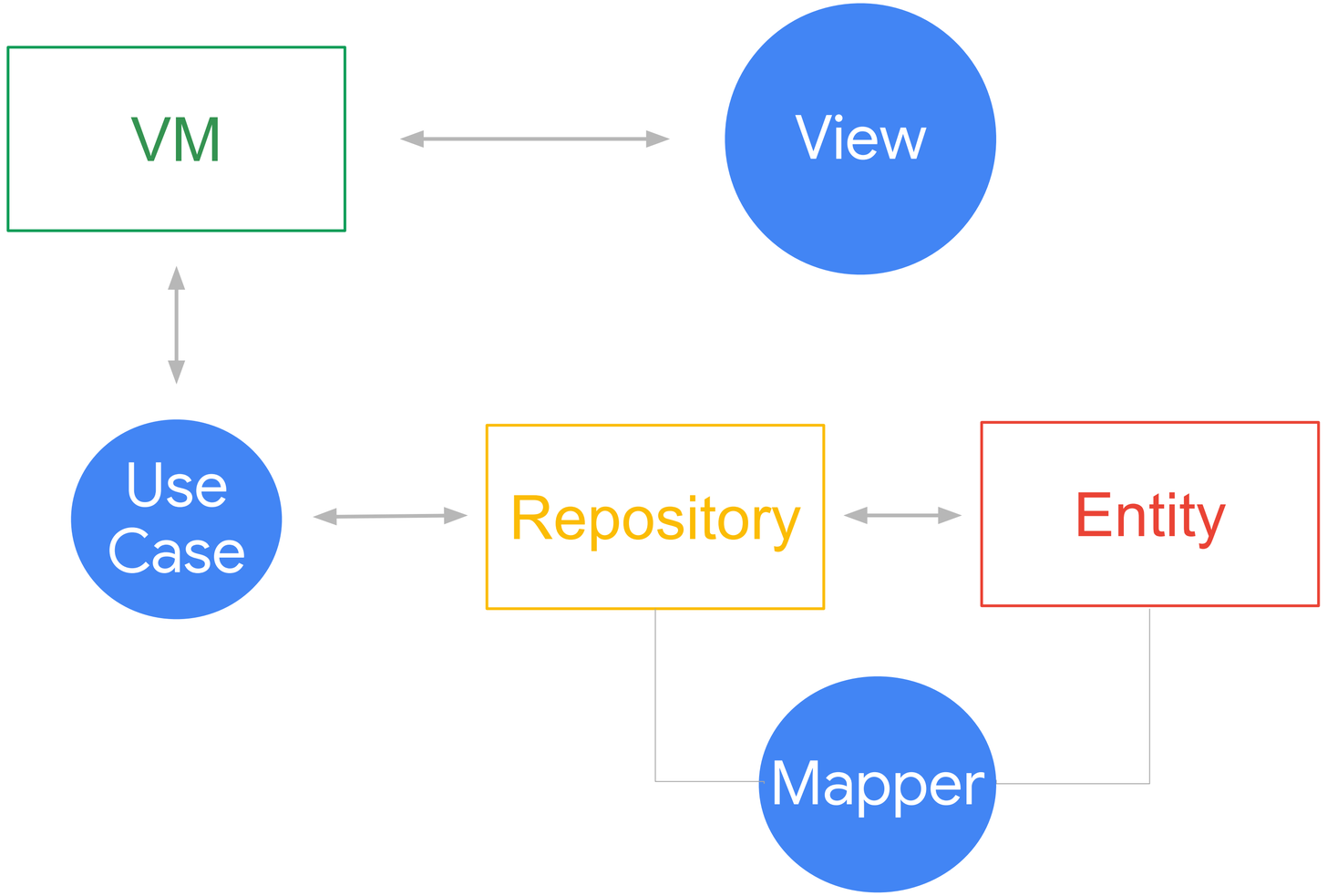
Experts

Decoupling Responsibility

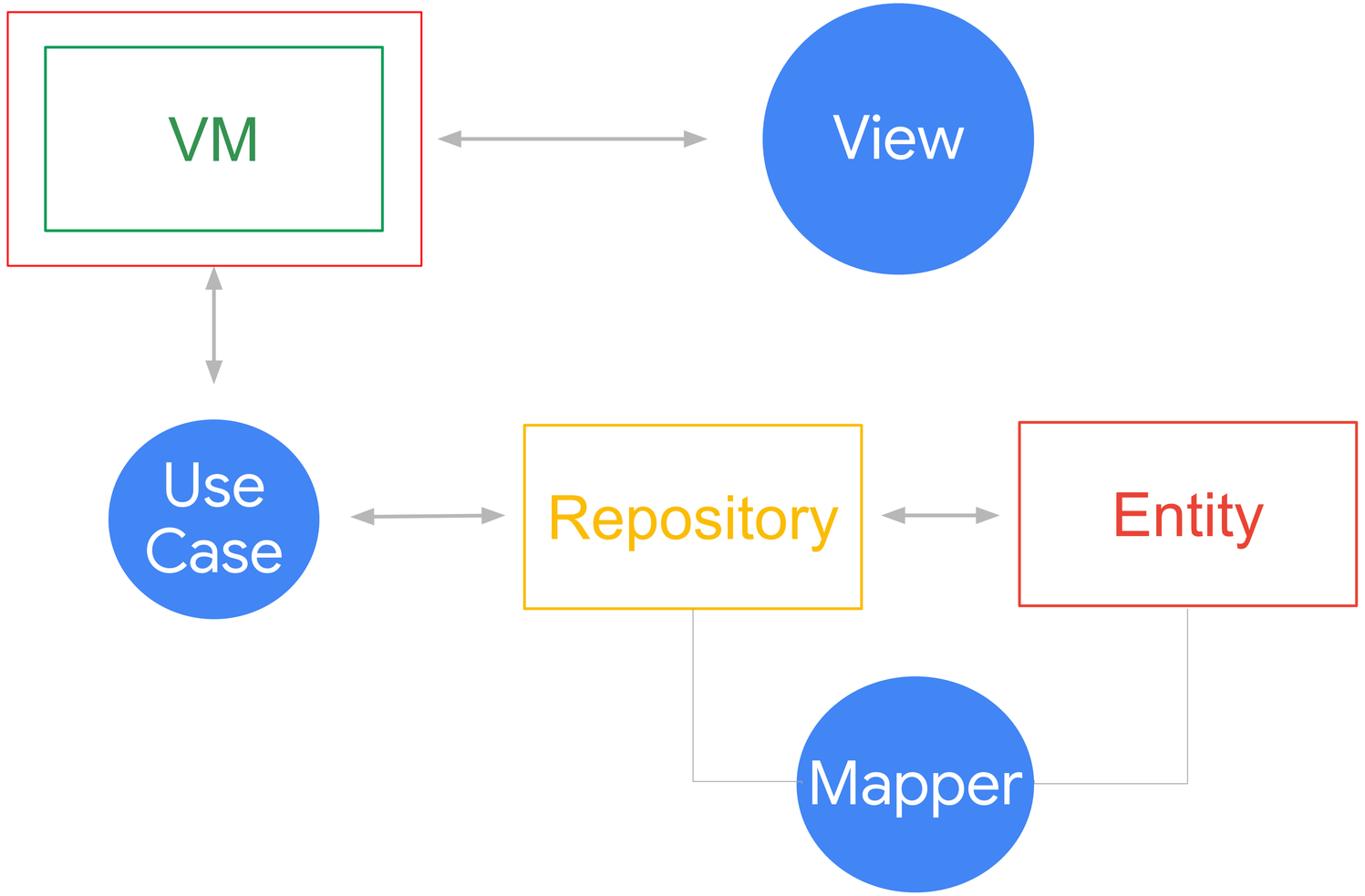
Knowing about Coroutines in the right layers



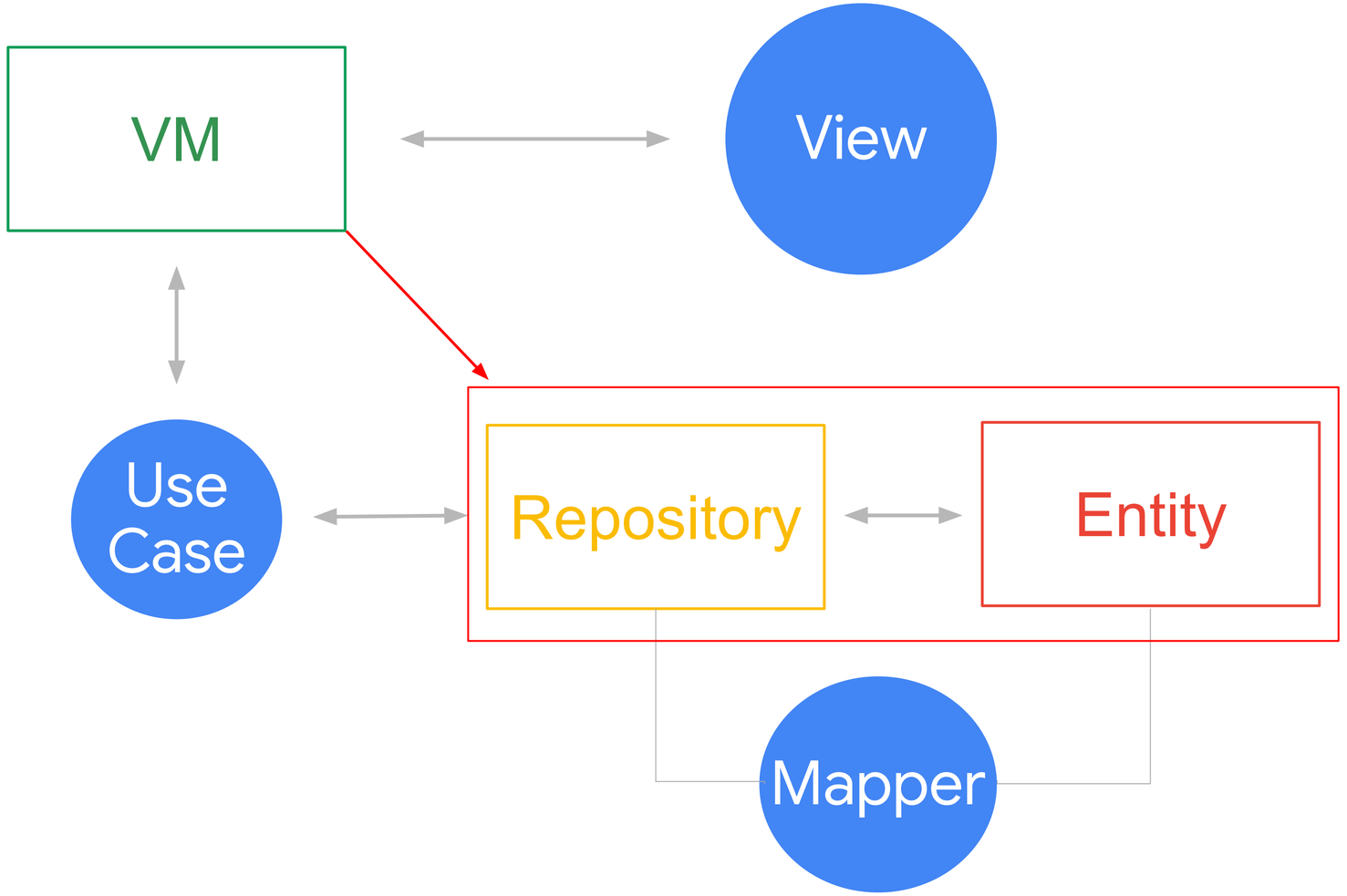
Layered approach



Layered approach



Layered approach



Moving Coroutines to Repositories

The Business layer shouldn't really know if the data comes from the **IO thread pool** the **Default pool**, or something else.

The business logic code should look and feel as if it were **sequential**, **non-asynchronous code**.

Additionally, sometimes, it’s best to rely on pre-built mechanisms!

```
// Room integrations with coroutines

@Dao
interface UsersDao {

    @Query("SELECT * FROM users")
    suspend fun getUsers(): List<User>

    @Insert
    suspend fun insertUser(user: User)

    @Delete
    suspend fun deleteUser(user: User)
}
```

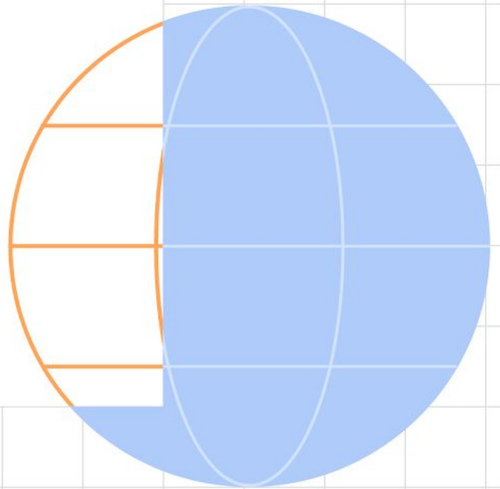
```
// Retrofit / Room integrations with coroutines

@GET // your path
fun getUserProfile(@Query(value = "id") userId: String): Call<UserProfile>

// With coroutines support
@GET
suspend fun getUserProfile(@Query(value = "id") userId: String): UserProfile
```

Resources

- Kotlin Coroutines By Tutorials
- Room & Coroutines
- Retrofit & Coroutines
- Coroutines on Android

Experts

Questions?



Google Developers

Experts

Thank You!

Filip Babić
@filbabic